Title: Compliance GPT AI-Powered Cybersecurity

PROJECT REPORT

Submitted By	Sabarna v
Roll Number	23sc043
college	Dr. N. G. P. Institute Of Technology
Organization	Payoda Technologies Private Limited

Table Of Contents

1. Project Overview

1.1 Project Introduction

ComplianceGPT is an AI-powered cybersecurity compliance assistant developed to make information retrieval from compliance documents easier and faster.The system allows users to upload documents such as PDF and text files and ask questions using a conversational interface. Instead of manually searching through large documents, users can ask questions in natural language and receive answers based on the information available in the uploaded documents.

The application uses a Retrieval-Augmented Generation (RAG) approach. Uploaded documents are processed, divided into smaller chunks, converted into embeddings, and stored in ChromaDB. When a user asks a question, the system retrieves relevant information using semantic and keyword-based search before sending the retrieved context to the LLM.The final response is generated using Llama 3.3 70B Versatile through the Groq API.

1.2 Problem Statement

Cybersecurity and compliance environments contain a large number of policies, procedures, standards, and guidelines. Manually searching these documents can be time-consuming, especially when the required information is located inside a large document.

Traditional keyword searches can also fail when the user and document use different words for the same concept.

ComplianceGPT addresses this problem by combining semantic retrieval, keyword retrieval, and LLM-based response generation to provide a more convenient way to access information from uploaded compliance documents.

1.3 Project Objectives

The main objective of the ComplianceGPT project is to provide a simple and user-friendly interface for uploading cybersecurity compliance documents and obtaining relevant information from them through natural-language queries. The system is designed to extract text from supported PDF and TXT documents and divide large documents into manageable chunks for efficient processing. These document chunks are converted into embeddings and stored along with the relevant document information in ChromaDB.

The project also aims to process user queries using Natural Language Processing (NLP) techniques and retrieve relevant information through a combination of semantic and keyword-based search. The retrieved information is then combined and provided as context to the Large Language Model (LLM), which generates a clear response based only on the available document context. This approach helps the system provide reliable document-based answers while reducing unsupported or hallucinated responses.

1.4 Proposed Solution

The proposed solution combines document processing, text chunking, embeddings, vector storage, NLP-based query processing, hybrid retrieval, and LLM generation to provide accurate answers from cybersecurity compliance documents. Uploaded documents are processed into smaller chunks, converted into embeddings, and stored in ChromaDB for efficient retrieval. When a user submits a question, the system processes the query and retrieves relevant information using semantic and keyword-based search. The retrieved context is then provided to the LLM, which generates a clear and document-based answer for the user.

The overall process is:

Document → Chunking → Embedding → ChromaDB → Query Processing → Retrieval → Context → LLM → Answer

2. Project Development Fundamentals

2.1 Python Development

Python is used as the primary programming language for the project.

It is used to implement:

- Document processing
- Embedding generation
- Database operations
- Query processing
- Retrieval
- RAG workflow
- Groq API communication
- Application logic

2.2 Project Structure

The project follows a modular structure where different responsibilities are separated into different Python modules.

Module	Responsibility
document_processor.py	Extracts, cleans and chunks documents
knowledge_manager.py	Manages embeddings and ChromaDB
query_processor.py	Processes user questions
hybrid_retriever.py	Performs semantic and keyword retrieval
groq_llm_manager.py	Communicates with Groq LLM
rag_engine.py	Connects retrieval and LLM generation
ingest.py	Ingests documents
check_docs.py	Checks stored documents
check_chromadb.py	Checks ChromaDB contents
direct_query.py	Tests direct ChromaDB retrieval
compat.py	Provides text-splitter compatibility

2.3 Virtual Environment

A Python virtual environment (venv) is used to keep project dependencies separated from other Python projects.

This prevents different projects from interfering with each other's package versions.

2.4 Environment Variables

The .env file is used to store configuration values separately from the source code.

The project uses settings such as:

- GROQ_API_KEY
- GROQ_MODEL
- GROQ_TEMPERATURE
- GROQ_MAX_TOKENS

This also prevents the API key from being directly written inside the application code.

2.5 Dependencies and Libraries

The project uses several Python libraries for different tasks.

Library	**Purpose**
Streamlit	Web interface
ChromaDB	Vector database
Sentence Transformers	Embedding generation
PyMuPDF	PDF text extraction
PyPDF2	PDF extraction fallback
spaCy	NLP processing
NLTK	NLP utilities
TextBlob	Sentiment analysis
scikit-learn	TF-IDF and cosine similarity
NumPy	Numerical operations
Groq	LLM API
python-dotenv	Environment configuration

3. Software Application Architecture

Stage 1 – Knowledge Creation / Document Ingestion

Document → DocumentProcessor → Chunks → KnowledgeManager → Embedding Model → ChromaDB.

Stage 2 – Query Processing and Retrieval

User Question → QueryProcessor → HybridRetriever → Relevant Chunks

Stage 3 – Answer Generation

Relevant Chunks → RAGEngine → Prompt + Context → GroqLLMManager → Groq API / Llama Model → Answer

Stage	Module / Component	Purpose	How It Works
Stage 1 – Knowledge Creation	DocumentProcessor	Process uploaded documents	Extracts text from documents and divides the text into smaller chunks.
	KnowledgeManager	Manage the knowledge base	Takes the document chunks, generates embeddings, and stores them in ChromaDB.
	SentenceTransformer	Generate embeddings	Converts each document chunk into a numerical vector representation.
	ChromaDB	Store document knowledge	Stores document chunks, embeddings, and metadata for later retrieval.
Stage 2 – Query Processing	QueryProcessor	Process the user question	Cleans the query and extracts keywords, phrases, entities, and related terms using NLP.
	HybridRetriever	Retrieve relevant information	Combines semantic search and keyword search to find the most relevant document chunks.
	Semantic Search	Meaning-based search	Converts the user query into an embedding and compares it with stored embeddings in ChromaDB.
	Keyword Search	Word-based search	Uses TF-IDF and cosine similarity to find chunks containing relevant keywords.
Stage 3 – Answer Generation	RAGEngine	Connect retrieval and answer generation	Combines the user's question with the retrieved document chunks and creates the prompt.
	GroqLLMManager	Communicate with the LLM	Sends the prepared prompt to the Groq API and receives the generated response.
	Groq API	Provide LLM access	Provides access to the configured language model for generating the response.
	Llama 3.3 70B	Generate final answer	Uses the provided context and question to generate the final human-readable answer.
Application	app.py	Provide the user interface	Provides the Streamlit interface for document upload, chat, and knowledge-base management.

4. Document Processing and Knowledge Management

4.1 Document Upload

The user uploads a supported document through the application.

Currently supported document types include:

- PDF
- TXT

The uploaded document is then passed into the document processing flow.

4.2 PDF and Text Extraction

For PDF files, the project primarily uses PyMuPDF (fitz) to extract text from each page.

PyPDF2 is used as a fallback if the primary extraction process fails.

For text files, the system attempts multiple encodings:

- UTF-8
- Latin-1
- CP1252

This helps the application handle text files with different encoding formats.

4.3 Text Cleaning

After extraction, unnecessary whitespace and unwanted special characters are removed.

This makes the extracted text cleaner before chunking and embedding.

4.4 Document Chunking

Large documents are divided into smaller pieces called chunks.

The current document processor uses:

- Chunk size: 2000 characters
- Chunk overlap: 200 characters

Chunking makes it easier for the retrieval system to find the specific section related to a user's question.

Simple idea:

Big document → Small searchable pieces

4.5 Chunk Size and Overlap

A chunk size of 2000 is the current configured value, not a permanent technical limit.

The value can be changed depending on the document and retrieval requirements.

The 200-character overlap helps preserve some information between neighboring chunks.

This reduces the chance of losing context when an important sentence occurs near a chunk boundary.

4.6 Metadata Generation

Each document chunk receives metadata such as:

- Source file name
- Chunk ID
- Chunk size
- File hash
- Word count
- File type
- Total number of chunks

This information helps identify and manage the retrieved document content.

4.7 Knowledge Manager

KnowledgeManager connects document processing, embedding generation, and ChromaDB. Its main flow is:

DocumentProcessor

↓

Valid Chunks

↓

Embedding Model

↓

Embeddings

↓

Metadata + IDs

↓

ChromaDB

5. Artificial Intelligence and Natural Language Processing

5.1 Artificial Intelligence

Artificial Intelligence (AI) is an important part of the ComplianceGPT system because it enables the application to provide natural-language responses to questions related to cybersecurity compliance documents. Instead of requiring users to manually search through lengthy policies and documents, the system allows them to ask questions in a simple conversational manner.

In ComplianceGPT, the AI component is mainly involved in generating the final response. Before the response is generated, the system retrieves relevant information from the uploaded documents. The retrieved document content is provided to the Large Language Model (LLM) as context along with the user's question. The LLM then uses this context to generate a clear and meaningful response.

The system is designed to restrict the LLM from generating unsupported information. The prompt provided to the model instructs it to use only the retrieved context and to indicate when sufficient information is not available. This approach helps improve the reliability of responses for cybersecurity compliance-related questions.

5.2 Natural Language Processing

Natural Language Processing (NLP) is used in ComplianceGPT to process and understand the user's question before retrieving relevant information. Users may ask questions in different forms, and therefore the query needs to be cleaned and analyzed before it is passed through the retrieval process.

The main NLP functionality is implemented through the QueryProcessor module. It performs several operations on the input query, including text cleaning, keyword extraction, key phrase extraction, entity extraction, query expansion, intent classification, question classification, and sentiment analysis.

For example, when a user asks:

"What are the password policy requirements?"

the QueryProcessor can identify important terms such as password, policy, and requirements. It can also expand the query with related compliance terminology to improve the chances of retrieving the appropriate document chunks.

The processed query is returned as structured information containing the original query, cleaned query, expanded query, key phrases, entities, intent, keywords, question type, and sentiment information.

5.3 spaCy

spaCy is an NLP library used by the QueryProcessor for analyzing the structure of the user's query. The project uses the en_core_web_sm English language model.After the query is cleaned, it is passed to spaCy for processing. The resulting document object is used to extract useful linguistic information.

spaCy is used for several purposes in the project:

- Key Phrase Extraction: Noun chunks are used to identify meaningful groups of words from the query.
- Entity Extraction: Named entities identified by spaCy are collected along with their labels and descriptions.
- Part-of-Speech Analysis: Words are analyzed based on their grammatical role, such as nouns, proper nouns, and adjectives.
- Lemmatization: Words can be represented using their base form, which helps normalize different forms of the same word.

5.4 NLTK

NLTK, or Natural Language Toolkit, is used as a supporting NLP library in the QueryProcessor. It provides utilities required for text preprocessing and linguistic analysis.The project uses English stop words to identify common words that generally provide less useful information during keyword extraction. These stop words are loaded and stored when the QueryProcessor is initialized.

NLTK's WordNetLemmatizer is also initialized as part of the query-processing setup. The project additionally checks for required NLTK datasets and downloads them when they are not already available.The use of NLTK together with spaCy provides supporting functionality for processing and preparing the user's query before retrieval.

5.5 TextBlob

TextBlob is used in the QueryProcessor for sentiment analysis of the user's query. Sentiment analysis provides two values: polarity and subjectivity.Polarity represents the general positive or negative orientation of the text, while subjectivity represents how strongly the text expresses an opinion or subjective viewpoint.

The project creates a TextBlob object from the user's query and extracts these values as part of the processed query information.Although sentiment is not the primary factor used for retrieving compliance documents, it is included as an additional feature of the query-processing pipeline. This allows the system to maintain a more complete representation of the user's input.

5.6 Query Cleaning

Query cleaning is the first major processing step performed on the user's question. Its purpose is to normalize the input and remove unnecessary formatting before further NLP operations are performed.

The QueryProcessor removes repeated whitespace and unwanted special characters while preserving words, spaces, and question marks.

5.7 Keyword and Entity Extraction

Keyword and entity extraction helps the system identify the important information contained in the user's question.During keyword extraction, the QueryProcessor focuses on words identified as nouns, proper nouns, or adjectives. Stop words are excluded, and the remaining words are converted into their lemma form where applicable.

The QueryProcessor also extracts entities identified by spaCy. Each entity is stored with its text, label, and description. This provides additional information about important named elements present in the query.

5.8 Query Expansion

Query expansion is used to improve retrieval by adding related cybersecurity and compliance terminology to the original query.

The QueryProcessor contains a predefined collection of compliance-specific terms and their related words. When a word in the user's query matches one of these terms or its related terminology, additional terms are added to the expanded query.

query expansion can be understood as:

User Query → Related Compliance Terms → Expanded Query → Better Retrieval Context

6. Retrieval-Augmented Generation (RAG)

6.1 RAG Overview

RAG stands for Retrieval-Augmented Generation.

Instead of directly asking the LLM to answer a question, the system first retrieves relevant information from the uploaded documents.

That information is then given to the LLM as context.

Memory trick:

RAG = Search first → Answer second

6.2 Embeddings

Embeddings convert text into numerical vectors.

For example:

"Password policy requires MFA"

↓

Embedding Model

↓

Numerical Vector

These vectors allow the system to compare the meaning of different pieces of text.

6.3 Sentence Transformer

The project uses the Sentence Transformers library to generate embeddings.

The specific model used is:

all-MiniLM-L6-v2

6.4 all-MiniLM-L6-v2

all-MiniLM-L6-v2 is used because it provides a practical balance between:

- Semantic understanding
- Speed
- Resource usage

It is used for both:

Document chunks → embeddings

User query → query embedding

Using the same model for both allows the system to compare them in the same vector space.

6.5 ChromaDB

ChromaDB is the project's vector database.

The collection is:

compliance_docs

The persistent storage location is:

chroma_db/

ChromaDB stores:

- Document chunks
- Embeddings
- Metadata
- IDs

6.6 Semantic Search

Semantic search uses embeddings to identify documents based on meaning rather than exact words.The query is converted into an embedding and compared with stored document embeddings.ChromaDB performs the vector search and returns relevant document chunks.

6.7 TF-IDF Keyword Search

The project also performs keyword-based retrieval using **TF-IDF**.

TF-IDF identifies important words in the query and checks how strongly they match the stored document chunks.

6.8 Cosine Similarity

Cosine similarity is used to measure how similar two vectors are.In the keyword retrieval stage, it is used to compare:

Query Vector

↓

Document Vectors

A higher similarity indicates a stronger match.

6.9 Hybrid Retrieval

The project combines:

Semantic Search + Keyword Search

Retrieval Method	Weight
Semantic Search	70%
Keyword Search	30%

This gives more importance to semantic meaning while still preserving exact keyword matching.

Memory trick:

Hybrid = Meaning + Words

6.10 Result Ranking

The results from both retrieval methods are combined.Duplicate chunks are removed, scores are combined, and the results are sorted from highest score to lowest score.The top required results are then returned to the RAG engine.

7. ComplianceGPT – AI-Powered Cybersecurity Compliance Assistant

7.1 Project Functionality

The main purpose of ComplianceGPT is to allow users to interact with their uploaded cybersecurity compliance documents through natural language.Instead of manually searching documents, the user can simply ask a question.

7.2 Document Ingestion

The ingest.py script searches the data folder for supported documents.

data/

- document1.pdf
- document2.txt

The documents are processed and added to the ChromaDB knowledge base.

7.3 Knowledge Base

The knowledge base contains the processed document chunks and their embeddings.

The persistent ChromaDB storage is:chroma_db/

The collection name is:compliance_docs

7.4 Query Flow

When the user asks a question:

User

↓

Streamlit

↓

RAGEngine

↓

QueryProcessor

↓

HybridRetriever

↓

Relevant Chunks

↓

LLM

↓

Answer

7.5 RAG Engine

RAGEngine is the central coordinator.

It connects:

- Knowledge Manager
- Query Processor
- Hybrid Retriever
- Groq LLM Manager

It retrieves the relevant document chunks and creates the context used by the LLM.

7.6 Groq API Integration

The project uses the **Groq API** to send the prompt and retrieved context to the selected LLM.The API key is loaded from the .env configuration rather than being directly written into the source code.

7.7 Llama 3.3 70B Versatile

The configured model is:Llama 3.3 70B Versatile.The model receives:

- System instructions
- Retrieved context
- User question

7.8 Prompt and Context Handling

The RAG engine creates a prompt that instructs the model to:

- Use only the supplied context.
- Avoid guessing.
- Avoid hallucinating information.
- State when sufficient information is unavailable.
- Provide clear and professional answers.

7.9 Streamlit Frontend

Streamlit is used to provide the user-facing web interface.It allows the user to interact with the application without directly interacting with the Python modules.

7.10 Response Generation

The final response follows:

User Question

↓

Prompt

↓

Groq API

↓

Llama 3.3 70B

↓

Final Respons

8. Project Testing and Validation

8.1 Document Verification

check_docs.py retrieves the documents currently available in the ChromaDB collection and displays their source names.It helps confirm whether the expected documents have been successfully added.

8.2 ChromaDB Verification

check_chromadb.py checks:

- Total number of chunks
- Chunks containing specific terms
- First few chunks in the knowledge base

This helps verify that document content has actually been stored.

8.3 Direct Query Testing

direct_query.py performs a direct ChromaDB query using:"password policy requirements"

It displays:

- Retrieved chunks
- Number of results
- Distance values

This is useful for checking whether ChromaDB retrieval is working correctly.

8.4 Retrieval Testing

The hybrid retriever can be tested by asking different questions and checking whether the retrieved chunks are relevant.

This validates both:

- Semantic retrieval
- Keyword retrieval

8.5 Response Validation

The generated response is checked against the uploaded document context.

The main objective is to ensure that the answer is based on the available knowledge base rather than unsupported information.

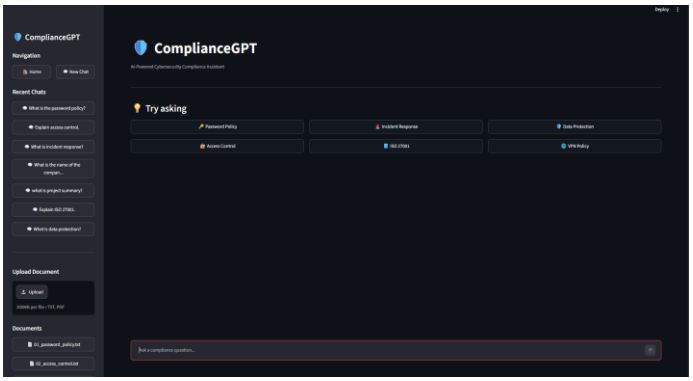
9. Results

All major features of the ComplianceGPT system were implemented and tested as part of the project. The application successfully provides a document-based cybersecurity compliance question-answering workflow using document processing, embeddings, ChromaDB, hybrid retrieval, NLP-based query processing, and LLM response generation

9.1 Application Screenshots

The following figures demonstrate the major user-facing features and important stages of the ComplianceGPT application.

Figure 9.1 – ComplianceGPT Main Interface



.

Figure 9.2 – Document Upload and Knowledge Base Interface

Document upload interface allowing users to add supported compliance documents to the knowledge base. The uploaded document is processed and indexed for later retrieval.The application displays available documents and provides knowledge-base management options. The implementation includes document listing, knowledge-base rebuilding, and clearing of stored documents.

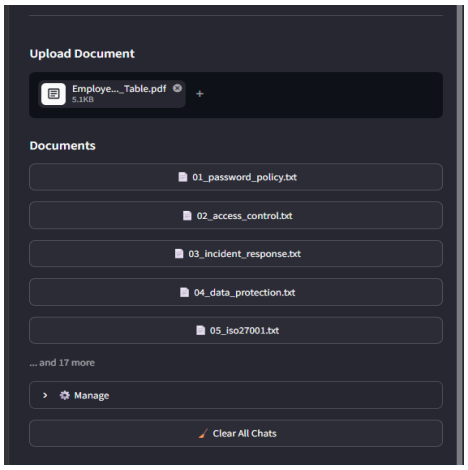


Figure 9.3 – Document Processing and Chunk Indexing

Successful document processing and indexing of the uploaded document into the knowledge base. The processed document is divided into chunks, embeddings are generated, and the resulting information is stored in ChromaDB.The application reports the number of chunks indexed after successful processing and refreshes the retriever so that the newly added information can be used for subsequent queries.

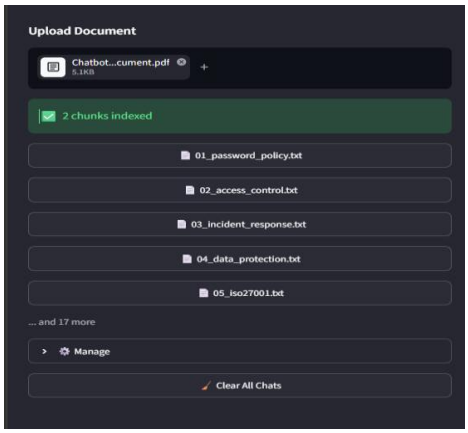


Figure 9.4 – Available Documents and Knowledge Base Management

Document management section displaying the documents currently available in the knowledge base along with options to rebuild the knowledge base or clear stored documents.The application reads the files available in the data/ directory and displays them in the interface.

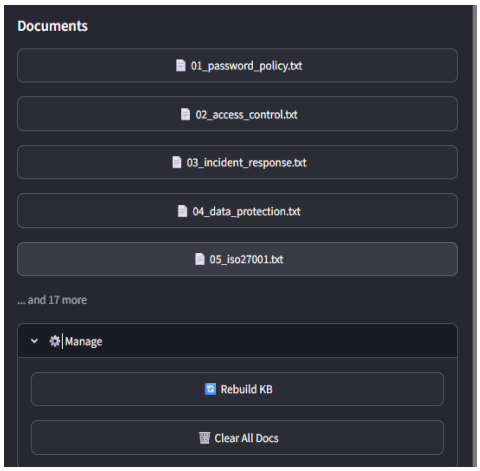


Figure 9.5 – Compliance Query Interface

Conversational query interface where users can enter natural-language questions related to cybersecurity compliance.The Streamlit application provides a chat input for compliance questions and displays the current conversation between the user and the assistant.

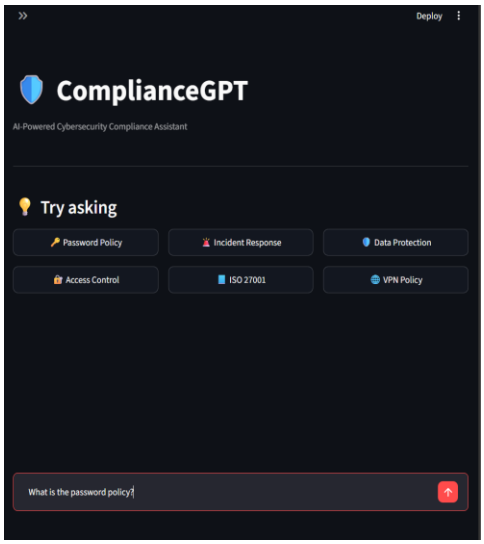


Figure 9.6 – Retrieved Context and AI-Generated Response

AI-generated response produced using the retrieved document context. The RAG workflow retrieves relevant information from the knowledge base and provides it to the LLM for response generation.The project architecture defines the retrieval and generation stages as separate steps, where relevant chunks are retrieved first and then used to generate the final answer.

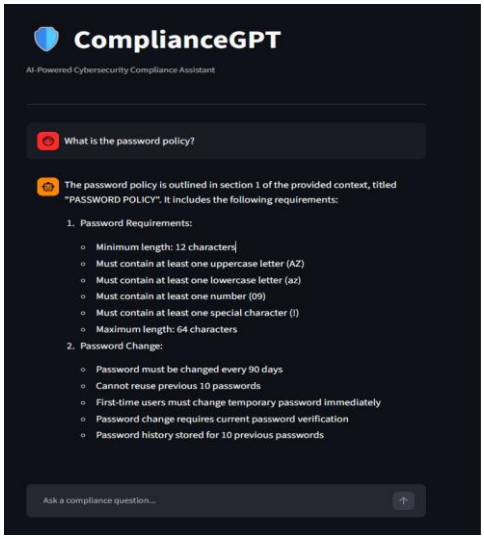


Figure 9.7 – ChromaDB Knowledge Base Verification

ChromaDB verification output displaying the stored document chunks and confirming that processed document content is available in the knowledge base.The project includes verification functionality for checking stored documents and chunks in the ChromaDB collection.

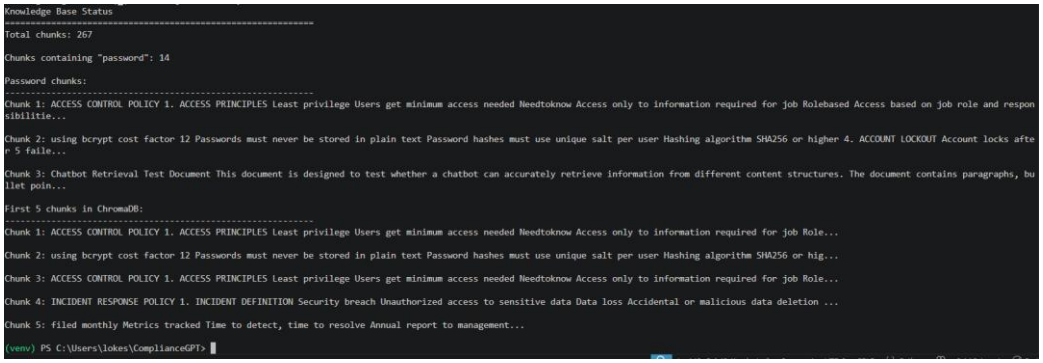


Figure 9.8 – Direct ChromaDB Query Testing

Direct ChromaDB query test demonstrating retrieval of relevant document chunks for a predefined compliance-related query.The direct query test can be used to verify whether ChromaDB is returning relevant chunks and to inspect the associated similarity distance values.

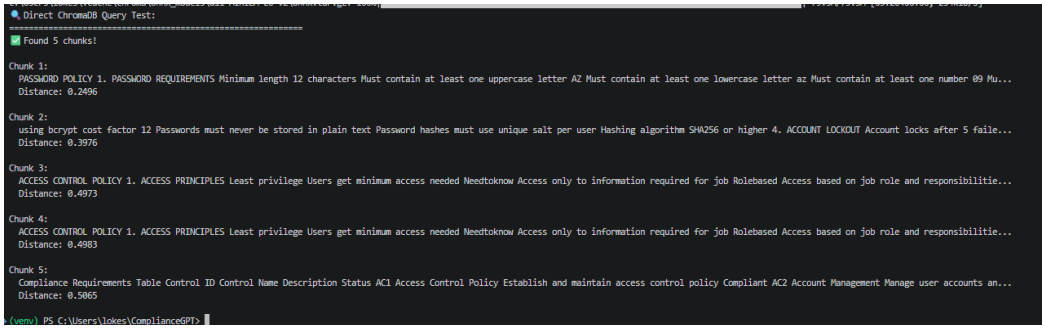
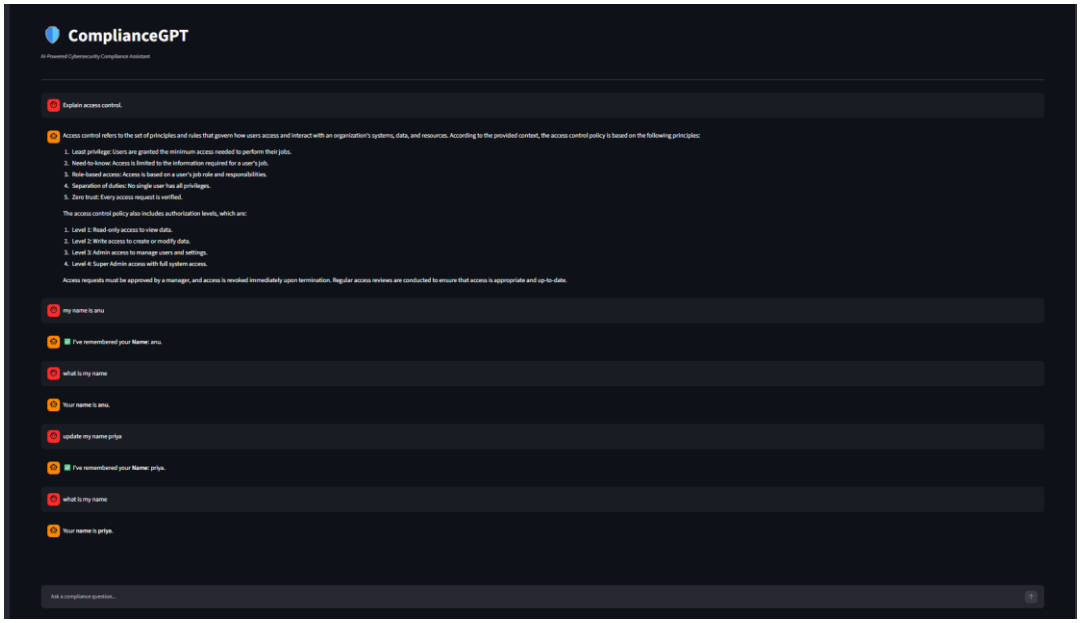


Figure 9.9 – Hybrid Retrieval and Query Processing

Query processing and hybrid retrieval workflow combining NLP-based query processing, semantic search, and keyword-based search to identify relevant document chunks.The hybrid retriever combines semantic results with TF-IDF keyword results and applies weighted scoring before returning the highest-ranked chunks.



.

10. Advantages and Limitation

Advantages

- Easy interaction through a conversational interface.
- Reduces manual document searching.
- Supports PDF and TXT documents.
- Uses semantic search.
- Uses keyword search.
- Combines both through hybrid retrieval.
- Uses persistent vector storage.
- Uses document context for answer generation.
- Modular project structure.

Limitations

- Answer quality depends on uploaded document quality.
- Retrieval quality depends on chunking and embedding quality.
- The application depends on Groq API availability.
- Table extraction support exists but is not part of the primary ingestion flow.
- The current knowledge base is limited to uploaded documents.

11. Future Enhancements

Future versions of the project can include:

1. Direct integration of table extraction into the document ingestion pipeline.
2. Better source citations for retrieved answers.
3. Advanced result reranking.
4. Support for additional document formats.
5. User authentication.
6. Role-based access control.
7. Retrieval performance monitoring.
8. Improved document management.
9. Larger organizational knowledge bases.
10. Better evaluation of retrieval and response accuracy.

12. Conclusion

ComplianceGPT demonstrates how AI, NLP, embeddings, vector databases, retrieval techniques, and large language models can be combined to simplify access to cybersecurity compliance information.

The system processes uploaded documents, creates searchable chunks, generates embeddings, stores them in ChromaDB, and retrieves relevant information when a user asks a question. Semantic and keyword retrieval are combined to improve the relevance of the retrieved information.

The retrieved context is then passed to Llama 3.3 70B Versatile through the Groq API, allowing the system to generate a natural-language response based on the available document information.

Overall, the project provides a practical foundation for building an AI-assisted compliance knowledge system and demonstrates the complete flow from document ingestion to context-based response generation.

Final technology summary

Category	Technology Used
Language	Python
Frontend	Streamlit
AI Approach	RAG
NLP	spaCy, NLTK, TextBlob
Document Processing	PyMuPDF, PyPDF2
Chunking	Custom document chunking
Embedding	Sentence Transformers
Embedding Model	all-MiniLM-L6-v2
Vector Database	ChromaDB
Semantic Retrieval	Embedding-based ChromaDB search
Keyword Retrieval	TF-IDF
Similarity	Cosine Similarity
Retrieval Strategy	Hybrid Retrieval
LLM Provider	Groq
LLM	Llama 3.3 70B Versatile
Configuration	.env
Document Source	PDF / TXT